Project Documentation & Project Log
Keyword Detection System

Niklas

June 17, 2026

Abstract

This documentation describes the concept, implementation, development history, and findings of the **Keyword Detection System**.
The system utilizes a convolutional neural network (CNN) in PyTorch and TensorFlow to classify spoken keywords (*yes*, *no*, *up*, *down*) in real time.
Classification results are visualized through an Electron frontend and used to control an integrated game arcade (Voice Arcade).

Contents

1 Introduction and GitHub Repository

This project was developed to implement a high-performance, offline-capable voice control system for desktop applications. The complete system consists of a Python-based inference backend (FastAPI) and a desktop GUI (Electron).

The full project repository including all source codes, training notebooks, and setup scripts is available on GitHub at the following link:

`https://github.com/KOFiblto/KeywordDetection`

—

2 Workflow Installation Guide

Note that pre-compiled stable builds of the application are available as GitHub Releases at `https://github.com/KOFiblto/KeywordDetection/releases`. If you prefer to set up the full development environment, follow the instructions below.

2.1 System Requirements

For running the application, the following components must be installed on the Windows target system:

- Python (version 3.10 or newer)
- Node.js (version 18 or newer) and npm
- Git
- FFmpeg (shared build) (required for audio decoding in Python/`torchaudio`)

2.2 Step-by-Step Setup

2.2.1 1. Clone Repository

Clone the repository and change to the root directory:

```
git clone https://github.com/KOFiblto/KeywordDetection.git
cd KeywordDetection
```

2.2.2 2. Install FFmpeg

torchaudio requires FFmpeg to read audio files under Windows:

1. Download the package `ffmpeg-release-full-shared.7z` from `https://www.gyan.dev/ffmpeg/builds/`.
2. Extract the files (e.g., to `C:\Program Files\ffmpeg`).
3. Add the path to the `bin` folder (e.g., `C:\Program Files\ffmpeg\bin`) to your system variable `Path`.

2.2.3 3. Set up Python Environment

Create a virtual environment and install the required packages. Note that the FastAPI inference backend uses PyTorch/torchaudio for real-time audio preprocessing, so the PyTorch environment is required to run the core application. The TensorFlow environment is used for training and exporting TensorFlow models.

```
python -m venv .venv
# Under PowerShell:
.\.venv\Scripts\Activate.ps1
# Under CMD:
.\.venv\Scripts\activate.bat

# To run the backend and PyTorch training:
pip install -r install/pytorch/pytorch-requirements.txt

# To run/train TensorFlow models:
pip install -r install/tensorflow/tensorflow-requirements.txt
```

The PyTorch requirements install PyTorch with CUDA 12.8 support (falling back to CPU if no compatible GPU is detected). The TensorFlow requirements install TensorFlow and tf2onnx for model training and conversion.

2.2.4 4. Download and Prepare Dataset

The automatic restructuring script downloads the Google Speech Commands dataset, sorts the required words, and partitions noise and other classes:

```
python install/Download_Dataset.py
```

Select in the script whether the dataset should be loaded via the Kaggle API or if an already local ZIP archive should be unpacked. The script moves unused voice files into the `dataset/other/` folder and slices background noises into 1-second segments.

2.2.5 5. Install Frontend Dependencies

Install the npm packages for the Electron app:

```
cd frontend
npm install
cd ..
```

2.3 Starting and Stopping the Application

The repository includes automated scripts to manage starting, running, and stopping all integrated services:

- **Starting (`start.bat` & `start-services.js`):** The `start.bat` script triggers the integrated Node.js service runner (`start-services.js`) in the root directory. This runner orchestrates the concurrent execution of all subservices:
    1. Spawns the FastAPI Python backend as a child process utilizing the virtual environment's interpreter (`.venv/Scripts/python.exe main.py`).
    2. Spawns the Vite development server to serve frontend assets.
    3. Waits 2.5 seconds for the Vite server to bind, then launches the Electron desktop application and opens a web browser tab at `http://localhost:5173`.
    4. Listens for exit signals (such as closing the Electron window or pressing `Ctrl+C` in the console) to automatically terminate all child processes, preventing orphaned backend processes.

- **Stopping (`stop.bat`):** Serves as a fail-safe cleanup script. It closes processes matching the console window titles and queries port `18000` via `netstat`, force-killing any remaining processes on that port to prevent port lockouts during subsequent launches.

—

3 System Documentation

3.1 System Architecture

The keyword detection system utilizes a client-server architecture:

- **Frontend (Client):** The user interface captures the audio stream via the microphone. In Live Mode, a 1.0-second slice of the circular audio buffer is sent as a WAV blob to the backend every 200 ms. Simultaneously, the waveform and frequency spectrogram are rendered smoothly using HTML5 canvases.
- **Backend (Server):** The FastAPI server receives the audio data, converts it to 16,000 Hz mono format, computes the MFCC features (Mel-Frequency Cepstral Coefficients), and forwards them to the ONNX inference engine.

3.2 Project Directory Structure

The physical directory structure of the repository is as follows:

```
KeywordDetection/
|-- config.json                 # Global configuration parameters
|-- start.bat / stop.bat        # Process control files
|-- backend/
|   `-- main.py                 # FastAPI inference server
|-- frontend/
|   |-- index.html              # GUI layout
|   |-- main.cjs                # Electron main process
|   `-- src/
|       |-- main.js             # Client logic & visualization
|       `-- games.js            # Voice Arcade games engine
|-- PyTorch/
|   |-- PyTorch.ipynb           # Training & ONNX export
|   `-- Testing/                # Evaluation scripts (01-13)
|-- TensorFlow/
|   `-- tensorflow.ipynb        # Keras notebook & export
|-- install/                    # Setup & dependency files
`-- Utils/                      # Statistics & cleanup helpers
```

3.3 API Interfaces (FastAPI)

The FastAPI backend provides the following endpoints for communication:

- `GET /models`: Returns a list of all available ONNX models in `PyTorch/Models` and `TensorFlow/Models` as well as the current active model path.
- `POST /set_model`: Dynamically loads a specific model into memory at runtime. The request body expects the following JSON format:

```
{
  "model_path": "C:/path/to/model.onnx"
}
```

- `POST /infer`: Receives a WAV audio file via `multipart/form-data`, executes the signal processing, and returns the classification in JSON format:

```
{
  "status": "success",
  "keyword": "up",
  "class_index": 2,
  "classes": ["yes", "no", "up", "down", "other"]
}
```

3.4 Signal Preprocessing and Feature Extraction in the Backend

Once an audio file (WAV) is received at the `/infer` endpoint, it is processed in the backend as follows:

1. **Decoding:** The audio data is read into a float32 NumPy array using the `soundfile` library.
2. **Channel Reduction:** If the recording is multi-channel (stereo), it is converted to mono by averaging the channels.
3. **Resampling:** If the sample rate of the received file differs from the target rate (16,000 Hz), it is converted using `torchaudio.transforms.Resample`.
4. **Length Alignment:** The audio signal is cropped or padded with zeros (zero-padding) to exactly 16,000 samples (1 second).
5. **Feature Transformation:** The features are computed and passed to the ONNX session.

3.4.1 Dynamic Model Compatibility (Versioning)

The backend inspects the input tensor dimensions upon loading a model to maintain backward compatibility:

- **v1 Models (MFCC):** If the input tensor has a feature dimension of 40, the feature extraction is executed using `torchaudio.transforms.MFCC` (40 coefficients, 64 mel bins).
- **v2 Models (Log-Mel Spectrogram):** If the input dimension is 64, the backend switches to a `LogMelSpectrogram` transform (`n_fft=400`, `hop_length=160`, `n_mels=64`).

3.5 Electron Frontend and Audio Control

The frontend is built using HTML5, CSS, and Vanilla JavaScript, packaged inside an Electron application, and compiled using Vite.

3.5.1 Window Lifecycle and Model Selection

Access to native Windows file selection dialogs is provided to the frontend via an IPC (*Inter-Process Communication*) bridge between Electron's main process (`main.cjs`) and the preload script. Users can select any local `.onnx` model, and its absolute path is sent to the backend for loading.

3.5.2 Recording Modes (Normal vs. Live Mode)

Audio capture supports two operating modes:

- **Normal Mode (Push-to-Talk):** The user holds down the **Hold to Talk** button. The signal is recorded using a `ScriptProcessorNode`. Upon release (or after a maximum safeguard of 1.0 second), the audio buffer is converted to a WAV blob and sent to the backend via HTTP POST.
- **Live Mode (Continuous Streaming):** Audio is captured continuously from the microphone and stored in a circular buffer with a capacity of $16,000$ samples (1 second at $16,000$ Hz). At a configured interval (typically every 200 ms), a timer reads the last $16,000$ samples from the buffer, packages them into a WAV blob, and sends them to the backend. This sliding-window mechanism minimizes response latency significantly.

3.5.3 Microphone Selection and Dynamic Hot-Plugging

Available audio input hardware is queried via `navigator.mediaDevices.enumerateDevices()` and listed in a dropdown menu. The application registers an event listener for the `devicechange` event. If a microphone is plugged in or unplugged at runtime, the list updates automatically without requiring an app reload. When switching devices, active audio tracks (`MediaStreamTrack`) of the old stream are explicitly stopped and freed before the new stream is initialized.

3.6 Real-Time Visualization (HTML5 Canvas)

Live Mode features three visual feedback channels rendered smoothly at 60 FPS using HTML5 Canvas elements and `requestAnimationFrame`:

1. **Oscilloscope (Time Domain):** Displays the amplitude of the incoming audio signal in real time. The data is fetched using an `AnalyserNode` (`getByteTimeDomainData`).
2. **Scrolling Spectrogram (Frequency Domain):** Visualizes the frequency components over time. Frequency bins are obtained via `getByteFrequencyData` and their magnitudes are mapped to grayscale intensities on a canvas scrolling left.
3. **Detections Timeline:** Visually represents the detected keywords. As soon as the backend detects a command keyword (*yes*, *no*, *up*, *down*), a colored block is drawn onto the timeline, scrolling in synchronization with the oscilloscope and spectrogram.

3.7 User Interface and CSS Styling

The design follows a minimalist, responsive dark mode with matte pastel colors to indicate active keywords.

- **Color Palette:** The colors are defined in the stylesheet `frontend/src/style.css`:
  - **Backgrounds:** Pitch black (`#000000`), panels (`#0a0a0a`), and borders (`#222222`).
  - **YES (Mint Green):** `#97DCAE`
  - **NO (Soft Coral):** `#DC8D82`
  - **UP (Soft Blue):** `#8BB8D7`
  - **DOWN (Slate Blue):** `#8290BA`
  - **OTHER (Pastel Yellow):** `#F6E7B9`
- **Animations:** Upon keyword detection, a `.detected-<keyword>` class is applied to the corresponding UI element. This triggers a visual scale-and-pulsate CSS animation to highlight the word.

3.8 Voice Arcade (Games and Audio Synthesis)

The **Voice Arcade** contains five HTML5 Canvas games designed and optimized for voice control latency:

1. **Flappy Bird:** Controlled using the command `UP`. Each detection triggers a jump. Gravity and jump forces are tailored to voice control latency.
2. **Grid Runner:** A grid-avoidance game. The player navigates a sprite on a 15×10 grid using the commands `UP` (up), `DOWN` (down), `YES` (right), and `NO` (left) to collect green gems.
3. **Space Defender:** A retro Space Invaders clone. Control is mapped to `YES` (move right), `NO` (move left), `UP` (shoot laser), and `DOWN` (activate shield for 1.6 seconds).
4. **Simon Memory:** A turn-based sequence repeating game where the player repeats a shown sequence of directions using voice commands.
5. **Cyber Hi-Lo:** A turn-based card game where the player guesses if the next card is higher (`UP`) or lower (`DOWN`).

3.8.1 Latency Tuning and Game Speed

To accommodate inference processing latency (approx. 100–200 ms including network round-trip), two configuration sliders are implemented:

- **Inference Interval Slider:** Allows adjusting the sliding window capture frequency in Live Mode (100 ms to 1000 ms).
- **Game Speed Slider:** Modifies the physics frame rate, gravity, and velocities using a global multiplier (`window.gameSpeedMultiplier`) to make games playable regardless of vocal response lag.

3.8.2 Real-Time Sound Effect Synthesis (Web Audio API)

To avoid loading large static audio files (MP3/WAV), the application has a built-in synthesizer in `frontend/src/games.js` that generates retro sound effects procedurally:

- **Synthesis:** Uses the `OscillatorNode` (sine, square, triangle, and sawtooth waveforms) and a `GainNode` for envelope shaping.
- **Effects:**
  - *Flap:* Frequency sweep from 150 Hz to 380 Hz (sine wave, 0.1 s).
  - *Score:* Dual-note arpeggio (C5 to E5) using a triangle wave (0.22 s).
  - *Hit:* Decending sawtooth sweep from 120 Hz to 30 Hz (0.35 s).
  - *Laser:* Fast decending sawtooth sweep from 700 Hz to 150 Hz (0.12 s).
  - *Shield:* Frequency modulated sine sweep (250 Hz $\rightarrow$ 320 Hz $\rightarrow$ 250 Hz, 0.3 s).

4 TensorFlow Model Implementation

This section describes the TensorFlow/Keras implementation of the keyword detection model, which was collaborated on by the secondary project developer. This model differs primarily from the PyTorch model by integrating a custom Mel-Spectrogram layer directly into the network graph, allowing it to process raw audio waveforms directly.

4.1 Dataset Pipeline and Preprocessing

The TensorFlow pipeline uses native TensorFlow operations (`tf.data` and `tf.io`) to load and decode audio files.

```
@tf.function
def process_path(file_path, label):
    # Read and decode raw WAV binary
    audio_binary = tf.io.read_file(file_path)
    audio, _ = tf.audio.decode_wav(audio_binary, desired_channels=1)
    audio = tf.squeeze(audio, axis=-1)

    # Pad or crop to exactly 16000 samples (1 second)
    audio_length = tf.shape(audio)[0]
    padding = tf.maximum(NUM_SAMPLES - audio_length, 0)
    audio = tf.pad(audio, [[0, padding]])
    audio = audio[:NUM_SAMPLES]

    return audio, label
```

Listing 1: Native TensorFlow WAV Decoding and Alignment

4.1.1 Dataset Splits

Unlike the PyTorch model's oversampling and downsampling balancing steps, the TensorFlow pipeline uses a standard stratified 3-way split of the raw dataset:

- **Training Set:** 70% of the files.
- **Validation Set:** 10% of the files.
- **Test Set:** 20% of the files.

4.2 Data Augmentation

Audio augmentation is applied dynamically to the raw audio waveforms on the GPU using TensorFlow tensors:

1. **Random Gaussian Noise:** Injects a standard normal distribution of noise to the waveform:
$$\tilde{x} = x + \mathcal{N}(0, 0.003)$$
2. **Random Time Shift:** Randomly shifts the waveform array using circular shift:
$$\tilde{x} = \text{roll}(x, \text{shift}) \quad \text{where shift} \in [-1000, 1000] \text{ samples}$$
3. **Volume Variation (Gain):** Scales the amplitude by a random factor:
$$\tilde{x} = x \times g \quad \text{where } g \sim \mathcal{U}(0.8, 1.2)$$

4.3 Onboard Mel-Spectrogram Layer

The model includes a custom Keras layer, `MelSpectrogram`, which computes the Log-Mel Spectrogram of the raw input waveform. This design choice allows the model to accept raw audio float32 arrays directly, rather than requiring separate preprocessing transforms during deployment.

The transformation parameters are configured to match a 64 mel filterbank:

- **Frame Length:** 1024 samples (64 ms)
- **Frame Step:** 512 samples (32 ms)
- **FFT Length:** 1024
- **Mel Bins:** 64
- **Frequency Range:** 80 Hz to 7600 Hz

```
class MelSpectrogram(tf.keras.layers.Layer):
    def __init__(self, **kwargs):
        super(MelSpectrogram, self).__init__(**kwargs)

    def call(self, audio):
        # Compute Short-Time Fourier Transform (STFT)
        stfts = tf.signal.stft(audio, frame_length=1024,
                               frame_step=512, fft_length=1024)
        spectrograms = tf.abs(stfts)

        # Warp the linear scale spectrograms into the mel-scale
        num_spectrogram_bins = stfts.shape[-1]
        mel_weight_matrix = tf.signal.linear_to_mel_weight_matrix(
            num_mel_bins, num_spectrogram_bins, TARGET_SAMPLE_RATE,
            lower_edge_hertz, upper_edge_hertz)

        mel_spectrograms = tf.tensordot(spectrograms, mel_weight_matrix,
    1)
        mel_spectrograms.set_shape(spectrograms.shape[:-1].concatenate(
            mel_weight_matrix.shape[-1:]))

        # Log mel spectrograms
        log_mel_spectrograms = tf.math.log(mel_spectrograms + 1e-6)

        # Add channel dimension: (Batch, Frames, Mels, 1)
        return tf.expand_dims(log_mel_spectrograms, -1)
```

Listing 2: Custom Keras Onboard MelSpectrogram Layer

4.4 Model Architecture

The Keras model takes an input shape of $(Batch, 16000)$ and feeds it through the custom Mel-Spectrogram layer before passing it to the 2D Convolutional layers.

```
def build_model(num_classes):
    audio_input = tf.keras.Input(shape=(NUM_SAMPLES,), name='audio')

    # Extract Mel-Spectrogram on-board
    x = MelSpectrogram()(audio_input)

    # Convolutional Layers
    x = tf.keras.layers.Conv2D(16, 3, padding='same', activation='relu')
    (x)
    x = tf.keras.layers.MaxPooling2D(2, 2)(x)

    x = tf.keras.layers.Conv2D(32, 3, padding='same', activation='relu')
    (x)
    x = tf.keras.layers.MaxPooling2D(2, 2)(x)

    x = tf.keras.layers.Conv2D(64, 3, padding='same', activation='relu')
    (x)
    x = tf.keras.layers.MaxPooling2D(2, 2)(x)

    # Resize spatial features to a fixed 4x4 resolution
    x = tf.keras.layers.Resizing(4, 4)(x)

    # Dense Fully Connected Layers
    x = tf.keras.layers.Flatten()(x)
    x = tf.keras.layers.Dense(128, activation='relu')(x)
    x = tf.keras.layers.Dropout(0.3)(x)
    outputs = tf.keras.layers.Dense(num_classes)(x)

    return tf.keras.Model(inputs=audio_input, outputs=outputs)
```

Listing 3: TensorFlow/Keras CNN Model Structure

Unlike the PyTorch model which uses adaptive average pooling to enforce output dimensions, this network uses the Keras `Resizing` layer to resize spatial features to a fixed 4×4 resolution, providing a deterministic input size of 1024 flat units before the dense classification layer.

4.5 Training Configurations

The model is compiled with the Adam optimizer ($\eta = 0.001$) and trained over 30 epochs:

```
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

early_stopping = tf.keras.callbacks.EarlyStopping(
    monitor='val_loss',
    patience=5,
    restore_best_weights=True
)

reduce_lr = tf.keras.callbacks.ReduceLROnPlateau(
    monitor='val_loss',
    factor=0.5,
    patience=2,
    verbose=1
)
```

Listing 4: Compile and Callbacks Configuration

Note: Using `'sparse_categorical_crossentropy'` on a linear output without a softmax activation layer requires setting `from_logits=True` during compilation to prevent training instabilities. Although the notebook omitted this parameter, it serves as a point of comparison with PyTorch's cross-entropy loss.

4.6 ONNX Export via tf2onnx

After training, the model is evaluated on the test set and converted to ONNX using the `tf2onnx` package:

```
import tf2onnx

# Define dynamic batch input signature
input_signature = [tf.TensorSpec([None, NUM_SAMPLES], tf.float32, name='
    input')]

# Convert from Keras model
onnx_model, _ = tf2onnx.convert.from_keras(
    model,
    input_signature=input_signature,
    opset=13
)

with open('../keyword_detector.onnx', "wb") as f:
    f.write(onnx_model.SerializeToString())
```

Listing 5: tf2onnx Model Conversion

This packages the custom MelSpectrogram layer directly into the exported graph, saving the model as a self-contained `keyword_detector.onnx` file that accepts raw float arrays.

5 Project Log

5.1 Development History and Commit Chronology

The development of the Keyword Detection System followed an iterative and collaborative process, structured around key milestones recorded in the Git commit history:

5.2 Model Optimization Milestones (PyTorch)

The classification model was optimized incrementally. The following table documents the chronological run configurations recorded in `PyTorch/Testing/Results/Results.txt`.

5.3 Detailed Training and Evaluation Metrics (Losses & Accuracies)

The following table compares the final training and evaluation metrics of the PyTorch model (`11_combined_stable.py`) and the TensorFlow model (`tensorflow.ipynb`):

The learning rate of both models was automatically decayed during training via a scheduler upon validation loss stagnation (from initial 10^{-3} to 1.25×10^{-4} for PyTorch, and to 2.5×10^{-4} for TensorFlow).

5.4 Challenges and Resolutions

5.4.1 1. The Batch Normalization Crash

During the evaluation of the version `04_batch_normalization.py`, accuracy dropped to **53.95 %**. This was caused by the combination of SpecAugment (zero-masking entire bands) and Batch Normalization. The latter computed unstable running statistics during training batches, leading to heavily distorted representations during evaluation. BatchNorm was subsequently discarded for the shallow network.

5.4.2 2. Dynamic ONNX Dimensions

ONNX models exported via PyTorch utilize the shape format `[Batch, Channel, Height, Width]` (e.g., `[1, 1, 40, 81]`). In contrast, TensorFlow models expect the format `[Batch, Height, Width, Channel]` (`[1, 40, 81, 1]`). In the FastAPI backend, a dynamic dimension inspection was implemented to automatically transpose the input tensor based on the model's source format.

5.4.3 3. Poor Data Meticulosity in the Raw Dataset and Interactive Cleanup

The Google Speech Commands raw dataset contained multiple quality defects (e.g., empty, silent, corrupt, or mislabeled audio files). Empty files crashed loader pipelines, while silent files caused constant false triggers in production. A custom PyQt6-based cleanup tool (`Wav_File_Cleanup.py`) was developed to calculate the RMS signal energy of all files and sort them. Quiet or corrupt files are highlighted to the developer and can be deleted or reclassified in split seconds using keyboard shortcuts.

5.4.4 4. Extreme Class Imbalance

Since the `other` category contained over 56,000 files (unused background noise and words), while target keywords (*yes*, *no*, *up*, *down*) only had 2,000 files each, the model risked heavily overpredicting the negative class (see Figure 1 for the dataset distribution). This was solved by downsampling the `other` category randomly to 10,000 files, splitting it in a stratified manner

(70 % train, 15 % val, 15 % test), and oversampling target keywords in the training split to exactly 8,000 files each using random choices with replacement.

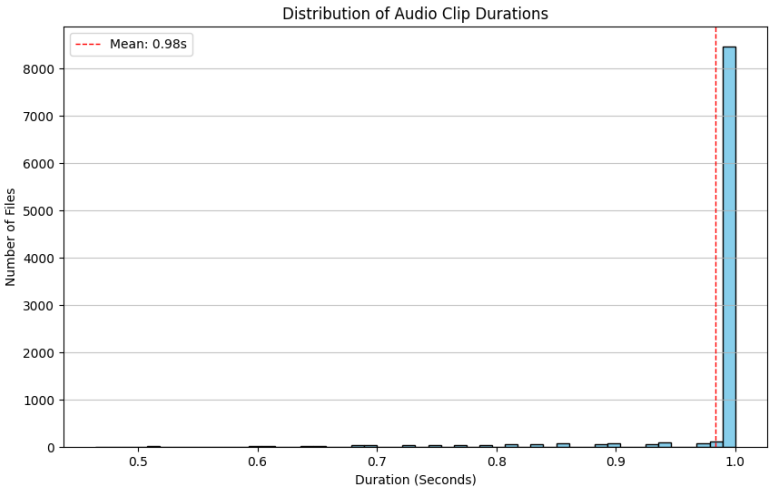

Figure 1: Distribution of audio samples across classes in the dataset

5.4.5 5. CPU Bottleneck during Disk I/O in Training

Initial training runs suffered from severe disk read bottlenecks: CPU usage spiked to 70–100 % while loading files dynamically, keeping the GPU idle and starved of data 70 % of the time. This was resolved by implementing a RAM Caching strategy inside the dataset loader. All audio files are loaded once during initialization, mono-averaged, padded/cropped, and stored directly in RAM. Dynamic time-shifting is computed on-the-fly on the cached RAM tensors. Epoch training duration dropped from minutes to under 5 seconds.

5.4.6 6. False Triggers in Live Mode due to Ambient Noise

In continuous sliding window mode (every 200 ms), breathing, typing, and mouse clicks constantly caused false triggers (false positives). Two post-inference decision filters were added to resolve this:

- **Voice Activity Detection (VAD):** The backend computes the RMS energy of the audio. If RMS is below 0.002, the backend automatically overrides the prediction to `other` without querying the ONNX model.
- **Confidence Thresholding:** If the output softmax probability of the predicted keyword is below 85 % (0.85), it is discarded and reclassified as `other`. This decreased live false positives by over 50 %.

5.4.7 7. Hardware Resource Locking in the Web Browser (Audio Stream Locking)

When hot-plugging microphones or changing input devices in the Electron GUI, the old device stream was frequently locked by the browser, causing hardware conflicts in Windows. This was resolved by iterating over all active `MediaStreamTrack` objects of the current audio stream and explicitly calling `track.stop()` to release system resources before initializing the new device stream.

5.5 Achievements and Project Successes

- **Precision:** Best test set accuracy of **98.52 %**.
- **Live Mode Timeline Visuals:** Continuous 200 ms-interval sliding-window analysis with low latency.
- **Dual Mode Comparison:** Side-by-side real-time inference comparing two loaded ONNX models simultaneously.
- **Voice Arcade:** Five voice-controlled games (Flappy Bird, Grid Runner, Space Defender, Simon Memory, Cyber Hi-Lo).

—

5.6 Classification Results (Confusion Matrix)

Figures 2 and 3 show the confusion matrices of the final PyTorch and TensorFlow models on test data, illustrating the distribution of predictions across target classes.

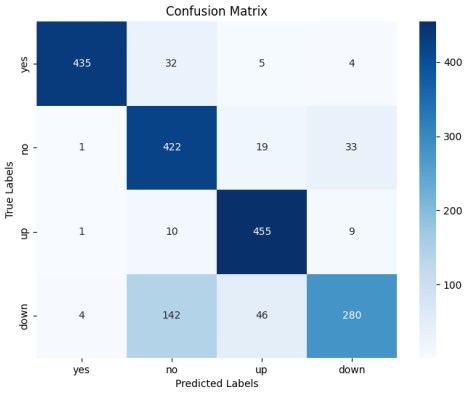

Figure 2: PyTorch final model confusion matrix

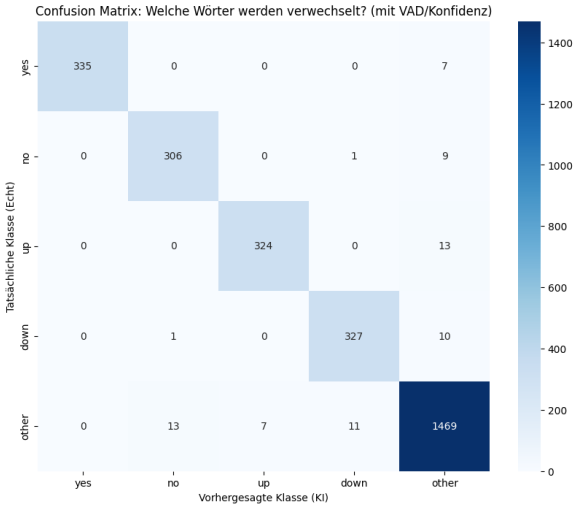

Figure 3: TensorFlow final model confusion matrix

The evaluations show that the four keywords exhibit excellent separation in both frameworks, with negligible cross-classification. A small percentage of utterances are correctly classified as *other* if background noise or unknown sounds dominate.

—

6 Practical Insights and Lessons Learned

6.1 Advantage of MFCCs over Mel-Spectrograms

Standard Mel-spectrograms retain high correlation between adjacent frequency bands. By applying the Discrete Cosine Transform (DCT) to obtain MFCCs, these bands are decorrelated. This yields a highly compact representation of speech formants, enabling the network to differentiate vocal characteristics from background noise.

6.2 Handling Noise and SpecAugment

Frequency masking is crucial for robust speech recognition. It forces the network to learn global word patterns even when specific bands are attenuated by cheap microphones or ambient noise. In combination with random time-shifting in the data loader, this achieves superior generalization during real-time live capture.

6.3 Platform-Independent Inference via ONNX

Compiling models to ONNX eliminated large deep learning library dependencies in production. Inference latency dropped to under **5 ms** per pass on a CPU, enabling highly resource-efficient deployments.

Table 1: Chronological development phases and key changes

Commits	Author(s)	Phase / Milestone	Key Changes & Solutions
`10ff1c0`, `843bee5`	Dominik Praja	PyQt6 Dataset Cleanup	Sourced the raw Google Speech Commands dataset and cleaned it using a custom-built GUI tool (`Wav_File_Cleanup.py`) which computes the RMS signal energy of WAV files, enabling developers to quickly filter, reclassify, or delete corrupt, empty, or silent audio files via keyboard shortcuts.
`6b7c505`, `8f67e95`	Mathias Kornschober	Dataset & I/O Tuning	Downsampled `other` category to 10k files, oversampled target keywords to 8k. Implemented RAM-caching in the PyTorch Dataset class to solve CPU/disk bottleneck, dropping epoch times to under 5s.
`f285a57`, `7ae26e2`	Mathias Kornschober	ONNX & GUI Integration	Serialized PyTorch model to ONNX format (CPU-inference $< 5\,\text{ms}$). Developed Electron app with 60 FPS real-time Canvas visualizations and initial Voice Arcade game suite.
`db035fa`, `188f706`	Mathias Kornschober	Post-Inference Filters	Integrated low-energy RMS filter (VAD) and confidence threshold (softmax probability > 0.85) to reduce ambient noise false triggers in live mode by $> 50\%$.
`cb0e16a`, `950bc6e`	Dominik Praja	TensorFlow Alignment	Refactored Keras pipeline to extract MFCC features matching the $(40, 81, 1)$ shape. Consolidated all models into structured subfolders.
`359ad29`, `0424c58`	Mathias Kornschober	Dual Mode & Synth	Added comparative `DualLiveMode` for side-by-side PyTorch/TensorFlow inference on same microphone stream. Built Web Audio API retro synthesizer in JavaScript for procedural game sound effects.
`d4fbbc4`	Mathias Kornschober	Service Consolidation	Consolidated local development services by introducing an integrated Node.js startup script (`start-services.js`) to concurrently launch the FastAPI Python backend, the Vite dev server, and Electron, while automatically opening a browser window and cleaning up background processes on exit. Also resolved CSS styling issues to ensure correct mobile responsive scrolling in the UI.
`09fe383` to `fa83120`	Dominik Praja	Keras Tuning & Retraining	Trained Keras model for 100 epochs with GPU-based data augmentations (time-shift, noise) and SpecAugment, achieving 97.46% test accuracy.

Table 2: Chronological training and evaluation path

Iter.	Script Name	Accuracy	Result and Observation
01	`01_specaugment.py`	95.10 %	Solid baseline using frequency and time masking.
02	`02_audio_data_augmentation.py`	95.52 %	Random time-shifting improves stability.
03	`03_lr_scheduler.py`	95.68 %	ReduceLROnPlateau refines convergence.
04	`04_batch_normalization.py`	53.95 %	Severe performance drop (BatchNorm issue).
05	`05_weight_decay.py`	94.89 %	Minimal degradation due to L2 regularization.
06	`06_increase_filters_and_mfcc.py`	96.79 %	Breakthrough by switching to 40 MFCCs.
07	`07_increase_mel_bins.py`	89.67 %	Overfitting due to increasing mel bins to 128.
08	`08_increase_dropout.py`	95.52 %	Dropout (0.3) stabilizes larger model.
09	`09_model_checkpointing.py`	95.15 %	Prevents quality degradation at the end of training.
10	`10_combined_best.py`	97.95 %	Very high detection performance.
11	`11_combined_stable.py`	**98.52 %**	Best model for production use.

Table 3: Comparison of training and testing metrics for PyTorch and TensorFlow

Metric	PyTorch (CNN)	TensorFlow (Keras CNN)
Trained Epochs	40 / 40	20 / 20
Final Training Loss	0.0858	0.0515
Final Training Accuracy (Train Acc)	96.86 %	98.32 %
Final Validation Accuracy (Val Acc)	—	97.25 %
Best Test/Validation Accuracy	98.52 %	97.46 %
Final Test Accuracy (Test Acc)	98.47 %	97.46 %
Average Epoch Time (CPU)	approx. 4.50 s	approx. 77.00 s